

Dental Feature – Frontend Reference

> Bu modul dental klinikalar üçündür. `appointments`, `customers`, `services` domenlərinə əlavə
> olaraq işləyir – köhnə strukturu dəyişmir. Hazırda business type gating tətbiq edilməyib; bütün
> hesablarda görünür. Gating: `currentBranch?.business\_type?.slug === 'dental'`.

1. Big Picture – Nə Lazımdır?

...

İstifadəçi (Dr. / Reception)

- └ Müştəri profilinə gedir → CustomerDetail modal
 - └ "Əməliyyatlar" tabı → PatientOperationsTab
 - └ Odontogram (diş seçimi)
 - └ Xidmət seçimi (real servislər kataloqdan)
 - └ Tarix seçimi (DatePicker)
 - └ Yadda Saxla → createAppointment + createService(s)
 - └ Keçmiş xidmətlər cədvəli (getPatientServices)
 - └ Diş tarixçəsi → getPatientOdontogram
 - └ Hansı dişlər müalicə edilib (sarı ilə göstərilir)

...

2. Cari Implementasiya – Nə Qurulub?

2.1 Fayl strukturu (mövcud)

...

src/

- └ shared/
 - └ components/
 - └ Odontogram/
 - └ Odontogram.tsx ← daimi dişlər (11-48)
 - └ OdontogramDeciduous.tsx ← süd dişlər (51-85)
 - └ index.tsx ← export barrel
 - └ services/
 - └ EndpointResources.g.ts ← dental namespace əlavə edilib
 - └ features/
 - └ dental/
 - └ components/
 - └ PatientOperationsTab/ ← əsas UI komponenti
 - └ index.tsx
 - └ patientOperationsTab.module.css
 - └ NewDentalVisitModal/ ← köməkçi modal (hal-hazırda istifadə olunmur)
 - └ hooks/
 - └ useOdontogramHistory.ts ← pasiyent tarixçəsi + refetch
 - └ services/
 - └ dental.api.ts ← DentalAPI static class
 - └ store/
 - └ dentalSession.store.ts ← Zustand, session persistence
 - └ types/
 - └ index.ts

...

3. Odontogram Kitabxanası – react-odontogram

```
```bash
npm i react-odontogram
```
```

3.1 Kritik Kəşf – Daxili ID Formatı

```
```pseudocode
// react-odontogram library source-dan:
// Tooth ID = `teeth-${quadrantPrefix}${toothPosition}`
// Məsələn: FDI tooth 26 → "teeth-26"
//
// defaultSelected prop YALNIZ bu formatla işləyir:
// DÜZGÜN: defaultSelected=["teeth-26", "teeth-18"]
// YANLIZ: defaultSelected=["26", "18"]} ← seçilmiş görünür!
//
// onChange callback isə aşağıdakı formatda qaytarır:
// d.id = "teeth-26"
// d.notations.fdi = "26" ← buradan FDI nömrəsini oxuyuruq
```

```typescript
// Buna görə Odontogram.tsx içində:
const toLib = (fdi: number) => `teeth-${fdi}`;

// defaultSelected:
defaultSelected={selected.map(toLib)} // ["teeth-26", "teeth-18"]

// teethConditions (prosedur rəngləri):
teeth: c.teeth.map((t) => toLib(Number(t))) // ["teeth-26"] ← düzgün
```
```

3.2 Süd Dişlər Mapping (FDI 51–85)

Library yalnız daimi diş ID-lərini (11–48 arası) dəstəkləyir. Süd dişləri üçün mapping lazımdır:

```
```pseudocode
// Quadrant mapping (süd → daimi):
// Süd quadrant 5 → Daimi quadrant 1 (üst sağ)
// Süd quadrant 6 → Daimi quadrant 2 (üst sol)
// Süd quadrant 7 → Daimi quadrant 3 (alt sol)
// Süd quadrant 8 → Daimi quadrant 4 (alt sağ)

FUNCTION decToPerm(deciduousId):
 quadrant = floor(deciduousId / 10) - 4 // 5→1, 6→2, 7→3, 8→4
 position = deciduousId % 10
 RETURN quadrant * 10 + position // 54 → 14, 71 → 31

FUNCTION permToDec(permanentId):
 quadrant = floor(permanentId / 10) + 4 // 1→5, 2→6, 3→7, 4→8
 position = permanentId % 10
 RETURN quadrant * 10 + position // 14 → 54, 31 → 71

// Axiş: OdontogramDeciduous.tsx
// 1. User clicks FDI 54 in library
// 2. Library onChange emits d.notations.fdi = "14" (mapped from internal)
// 3. We apply permToDec(14) = 54 → store 54 in entries
// 4. For display: decToPerm(54) = 14 → pass "teeth-14" to defaultSelected
```
```

3.3 Rəng Sistemi – teethConditions

```
```pseudocode
// Hər prosedur/xidmət özünün rənginə sahib olur:
PALETTE = [
{ fill: "#DBEAFE", outline: "#2563EB" }, // mavi
{ fill: "#D1FAE5", outline: "#059669" }, // yaşıl
{ fill: "#FEE2E2", outline: "#DC2626" }, // qırmızı
... // 8 rəng
]

// PatientOperationsTab prosedur rəng hesablaması:
FUNCTION buildProcedureConditions(entries):
colorMap = Map<serviceId, colorIndex>
groups = Map<serviceId, toothIds[]>

FOR EACH entry IN entries WHERE entry.serviceId EXISTS:
IF serviceId NOT IN colorMap:
colorMap[serviceId] = colorMap.size // ilk rast gəlinən xidmət → 0, sonrakı → 1,
...
groups[serviceId].push(entry.toothId)

RETURN groups.map((serviceId, toothIds) =>
idx = colorMap[serviceId] % 8
{ label, teeth: toothIds.map(toLib), fillColor: PALETTE[idx].fill, outlineColor:
PALETTE[idx].outline }
)

// CSS necə işləyir:
// defaultSelected dişlər → library default mavi (CSS .selected class)
// teethConditions dişlər → data-colored="true" → opacity: 1 →
condition.fillColor görünür
// ikisi birlikdə: condition.fillColor üstünlük alır (inline fill overrides CSS
fill)
```

---
```

4. State Persistence – useDentalSessionStore

Modal bağlandıqda seçimlər itməsin deyə Zustand session store istifadə olunur:

```
```pseudocode
// store/dentalSession.store.ts
STATE:
byCustomer: Record<customerId, {
perm: ISessionEntry[] // daimi dişlər üçün seçimlər
dec: ISessionEntry[] // süd dişləri üçün seçimlər
}>

ISessionEntry:
toothId: number // FDI nömrəsi
serviceId: number|null // real servis ID (ServicesAPI-dan)
serviceName: string // göstərmək üçün
date: string|null // ISO string (Dayjs serialized)

ACTIONS:
getEntries(customerId, view: 'perm'|'dec') → ISessionEntry[]
setEntries(customerId, view, entries) // hər seçim dəyişikliyinə
clear(customerId) // save sonrası

// Komponentdə REACTIVE abunəlik:
// :white_check_mark: DÜZGÜN – yenidən render olur:
```

```
entries = useDentalSessionStore(s => s.byCustomer[customerId]?.[storeView] ??
[])
```

```
// :x: YANLIZ – reaktiv deyil:
const { getEntries } = useDentalSessionStore(s => s.actions)
entries = getEntries(customerId, storeView) // yalnız ilk render-də oxunur!
`
```

---

## 5. PatientOperationsTab – İş Axış 1 (Pseudocode)

```
`pseudocode
// MOUNT:
FUNCTION onMount():
ServicesAPI.getBranchServices() → branchServices (real xidmətlər kataloqu)
useOdontogramHistory(customerId) → { services, treatedTeeth, treatedDeciduous }
```

```
// DİŞ SEÇİMİ:
FUNCTION handleChange(newlySelectedTeeth: number[]):
prevIds = entries.map(e => e.toothId)
added = newlySelectedTeeth.filter(id => NOT IN prevIds)
// NOT: yalnız yeni dişlər əlavə edilir, silinmir (toggle yox, yalnız add)
// Silmə yalnız x düyməsi ilə olur
saveEntries([...entries, ...added.map(id => {toothId: id, serviceId:
null, ...})])
```

```
// XİDMƏT VƏ TARİX TƏYİNİ (Popover içərisində):
FUNCTION handleProcedureChange(toothId, serviceId, serviceName, date):
saveEntries(entries.map(e =>
e.toothId === toothId
? { ...e, serviceId, serviceName, date: date.toISOString() }
: e
))
```

```
// YADDA SAXLA:
FUNCTION handleSave():
toSave = entries.filter(e => e.serviceId EXISTS)
IF toSave.isEmpty:
show warning "ən azı bir xidmət seçin"
RETURN
```

```
TRY:
appt = await DentalAPI.createAppointment({
name: "Diş müalicəsi – {today}",
customer_id: customerId,
employee_id: profile.id, // cari istifadəçi (həkim)
duration: 30, // default
reserve_at: now.toISOString()
})
appointmentId = appt.data.id
```

```
await Promise.all(
toSave.map(entry =>
DentalAPI.createService(appointmentId, {
tooth_numbers: String(entry.toothId), // "26"
service_id: entry.serviceId,
date: entry.date ?? now,
price: branchServices.find(s => s.id === serviceId).start_price ?? 0
})
)
)
```

```
show success notification
```

```
store.clear(customerId) // seçimlər silinir
refetch() // cədvəl yenilənir
```

```
CATCH error:
show error notification
```
```

6. Backend Endpoint-ləri

> Konvensiya: `/api/` prefix yoxdur. `EndpointResources.g.ts`-ə əlavə et – başqa yerdə hardcode etmə.

6.1 EndpointResources.dental namespace

```
```typescript
// src/shared/services/EndpointResources.g.ts
public static readonly dental = {
 appointments: '/dental/appointments',
 appointment: (id: number) => `/dental/appointments/${id}`,
 appointmentStatus: (id: number) => `/dental/appointments/${id}/status`,
 appointmentServices: (apptId: number) =>
 `/dental/appointments/${apptId}/services`,
 appointmentService: (apptId: number, svcId: number) => `/dental/appointments/${
 apptId}/services/${svcId}`,
 patientOdontogram: (customerId: number) =>
 `/dental/patients/${customerId}/odontogram`,
 patientServices: (customerId: number) =>
 `/dental/patients/${customerId}/appointment-services`,
};
```
```

6.2 Dental Appointments CRUD

`POST /dental/appointments` – Yeni vizit yarat

...

REQUEST BODY:

```
{
  name: string // "Diş müalicəsi – 12.07.2026"
  customer_id: number // required – pasiyent
  employee_id: number // required – həkim (profile.id)
  duration: number // dəqiqə, default: 30
  reserve_at: string // ISO datetime, default: now
  note?: string
}
```

RESPONSE 201:

```
{
  result: true,
  data: {
    id: number // ← createService üçün lazım olan appointmentId
    name, customer_id, employee_id, duration, reserve_at, status, services[],
    created_at
  }
}
```
```

#### ##### `GET /dental/appointments` – Siyahı

...

QUERY PARAMS (hamısı optional):

branch\_id, date\_from, date\_to, employee\_id, status, customer\_id, page, limit

RESPONSE:

```
{
 result: true,
 data: IDentalAppointment[],
 pagination: { current, prev, next, total, totalPages }
}
```

#### `GET /dental/appointments/:id` – Tək vizit

...

RESPONSE: { result: true, data: IDentalAppointment } // services[] populate edilmiş

...

#### `PUT /dental/appointments/:id` – Yenilə

...

BODY (bütün fieldlər optional): name?, employee\_id?, duration?, reserve\_at?, note?

...

#### `PUT /dental/appointments/:id/status` – Status

...

BODY: { status: "PENDING"|"BOOKED"|"START"|"COMPLETED"|"CANCEL", reason?: string }

...

#### `DELETE /dental/appointments/:id`

...

RESPONSE: { result: true, data: null }

...

---

### ### 6.3 Appointment Services CRUD

#### `POST /dental/appointments/:id/services` – Xidmət əlavə et

...

REQUEST BODY:

```
{
 tooth_numbers: string // "26" – bir diş üçün. Birdən çox: "26,27"
 service_id: number // ServicesAPI-dan gələn real xidmət ID-si
 date: string // ISO datetime
 price: number // xidmət qiyməti (service.start_price)
 expenses?: number // default: 0
 discount?: number // default: 0
 discount_auto?: boolean // default: false
 executor_percentage?: number // default: 0
 executor_amount?: number // backend hesablaya bilər (price * pct / 100)
 referrer_percentage?: number // default: 0
 referrer_amount?: number
 note?: string
 employee_id?: number // icraçı (fərqli həkim ola bilər)
 referrer_id?: number
}
```

RESPONSE 201: { result: true, data: IAppointmentService }

```
// Hazırkı implementasiyada hər diş üçün ayrıca createService çağırılır:
// entries.forEach(e => createService(apptId, { tooth_numbers:
String(e.toothId), ... })))
// Alternativ: bir çağırışda: { tooth_numbers: "26,27,18", service_id: X }
```

```
`GET /dental/appointments/:id/services`
```

```
...
```

```
RESPONSE: { result: true, data: IAppointmentService[] }
```

```
...
```

```
`PUT /dental/appointments/:apptId/services/:serviceId`
```

```
...
```

```
BODY: Partial<IAppointmentServiceBody> // yalnız dəyişən fieldlər
```

```
...
```

```
`DELETE /dental/appointments/:apptId/services/:serviceId`
```

```
...
```

```
RESPONSE: { result: true, data: null }
```

```
...
```

```

```

### ### 6.4 Pasiyent Tarixçəsi

```
`GET /dental/patients/:customerId/odontogram` – Diş vəziyyəti
```

```
...
```

```
RESPONSE:
```

```
{
 result: true,
 data: {
 customer_id: number,
 teeth: [
 {
 tooth_number: number // FDI 11-85 (daimi 11-48, süd 51-85)
 status: string // "TREATED" | "MISSING" | "PLANNED"
 last_treated_at: string|null
 appointment_id: number|null
 service_name: string|null
 }
]
 }
}
```

```
// Frontend istifadəsi (useOdontogramHistory.ts):
```

```
treatedTeeth = teeth.filter(t => t.tooth_number >= 11 && t.tooth_number <= 48)
```

```
treatedDeciduous = teeth.filter(t => t.tooth_number >= 51 && t.tooth_number <= 85)
```

```
// Odontogram-a history prop kimi ötürülür → sarı/narıncı rənglə göstərilir
```

```
...
```

```
`GET /dental/patients/:customerId/appointment-services` – Bütün xidmətlər
```

```
...
```

```
// PatientOperationsTab-dakı tarixçə cədvəli bu endpoint-dən doldurulur
```

```
RESPONSE: { result: true, data: IAppointmentService[] }
```

```
...
```

---

## ## 7. TypeScript Tip Definisiyaları

```
``typescript
// src/features/dental/types/index.ts

export enum ToothStatus {
 TREATED = 'TREATED', // yaşıl
 MISSING = 'MISSING', // kırmızı / çıkarılmış
 PLANNED = 'PLANNED', // mavi / planlaştırılmış
}

export interface IToothRecord {
 tooth_number: number;
 status: ToothStatus;
 last_treated_at: string | null;
 appointment_id: number | null;
 service_name: string | null;
}

export interface IPatientOdontogram {
 customer_id: number;
 teeth: IToothRecord[];
}

export interface IAppointmentService {
 id: number;
 appointment_id: number;
 tooth_numbers: string; // "26" or "26,27,18"
 service_id: number;
 service: {
 id: number;
 name: string;
 price: number;
 category: { id: number; name: string } | null;
 };
 date: string;
 price: number;
 expenses: number;
 discount: number;
 discount_auto: boolean;
 total_amount: number;
 executor_percentage: number;
 executor_amount: number;
 referrer_percentage: number;
 referrer_amount: number;
 note: string | null;
 employee_id: number | null;
 employee: { id: number; name: string; surname: string | null } | null;
 referrer_id: number | null;
 referrer: { id: number; name: string; surname: string | null } | null;
 created_at: string;
 is_completed: boolean;
}

export interface IDentalAppointment {
 id: number;
 name: string;
 customer_id: number;
 employee_id: number;
 duration: number;
 reserve_at: string;
 note: string | null;
}
```



```

status: string;
services: IAppointmentService[];
created_at: string;
}

```

```

export interface IDentalAppointmentBody {
name: string;
customer_id: number;
employee_id: number;
duration: number;
reserve_at: string;
note?: string;
}

```

```

export interface IAppointmentServiceBody {
tooth_numbers: string;
service_id: number;
date: string;
price: number;
expenses?: number;
discount?: number;
discount_auto?: boolean;
executor_percentage?: number;
executor_amount?: number;
referrer_percentage?: number;
referrer_amount?: number;
note?: string;
employee_id?: number;
referrer_id?: number;
}
...

```

---

### ## 8. Session Store – ISessionEntry

```

```typescript
// src/features/dental/store/dentalSession.store.ts

export interface ISessionEntry {
toothId: number; // FDI nömrəsi (11-85)
serviceId: number | null; // ServicesAPI-dan gələn real ID
serviceName: string; // UI-da göstərmək üçün
date: string | null; // dayjs().toISOString()
}

// Zustand store – byCustomer[id].perm / byCustomer[id].dec
// Modal bağlanıb açılanda seçimlər QALIR (session ərzində)
// Save sonrası store.clear(customerId) çağırılır
...

```

9. Servislər Kataloqu – Xidmət Seçimi

```

```pseudocode
// PatientOperationsTab mount-da:
ServicesAPI.getBranchServices({ limit: 200, page: 1 })
→ res.data = [{ id, name, services: [{ id, name, start_price, ... }] }]
→ flat = data.flatMap(cat => cat.services ?? [])
→ setBranchServices(flat)

// ToothEntryRow Popover-da:
<Select

```

```
options={branchServices.map(s => ({ label: s.name, value: s.id })))}
// user seçir → serviceId + serviceName entries-ə yazılır
/>

// Save-də qiymət:
price = branchServices.find(s => s.id === entry.serviceId)?.start_price ?? 0
`
```

---

## ## 10. FDI Nömrələmə Sistemi

...

YUXARI ÇƏNƏ (üst) – xəstənin baxışı

SAĞ ÜST (Q1)	SOL ÜST (Q2)	
Daimi: 18 17 16 15 14 13 12 11		21 22 23 24 25 26 27 28
Süd: 55 54 53 52 51		61 62 63 64 65

Süd: 85 84 83 82 81		71 72 73 74 75	
Daimi: 48 47 46 45 44 43 42 41		31 32 33 34 35 36 37 38	
SAĞ ALT (Q4)	SOL ALT (Q3)		

AŞAĞI ÇƏNƏ (alt)

Diş tipi:

Daimi: 11-18, 21-28, 31-38, 41-48 (cəmi 32 diş)

Süd: 51-55, 61-65, 71-75, 81-85 (cəmi 20 diş)

Quadrant prefix:

1x = Sağ üst daimi 5x = Sağ üst süd

2x = Sol üst daimi 6x = Sol üst süd

3x = Sol alt daimi 7x = Sol alt süd

4x = Sağ alt daimi 8x = Sağ alt süd

...

---

## ## 11. useOdontogramHistory Hook

```typescript

// src/features/dental/hooks/useOdontogramHistory.ts

// Nə edir:

// 1. getPatientOdontogram → treatedTeeth, treatedDeciduous

// 2. getPatientServices → xidmətlər cədvəli üçün

// 3. refetch() → tick artırır → useEffect yenidən işləyir

FUNCTION useOdontogramHistory(customerId):

[tick, setTick] = useState(0)

refetch = () => setTick(t => t + 1)

ON [customerId, tick] CHANGE:

setLoading(true)

[odontogramRes, servicesRes] = await Promise.all([

DentalAPI.getPatientOdontogram(customerId),

DentalAPI.getPatientServices(customerId),

])

setOdontogram(odontogramRes.data)

setServices(servicesRes.data)

setLoading(false)

treatedTeeth = odontogram.teeth.filter(t => 11 ≤ t.tooth_number ≤ 48).map(t =>

```
t.tooth_number)
treatedDeciduous = odontogram.teeth.filter(t => 51 ≤ t.tooth_number ≤ 85).map(t
=> t.tooth_number)
```

```
RETURN { odontogram, services, treatedTeeth, treatedDeciduous, loading,
refetch }
```
```

---

## ## 12. Ümumi Response Formatı

```
```typescript
// Bütün dental endpoint-lər:
interface IResponse<T> {
result: boolean;
data: T;
message?: string;
pagination?: { current, prev, next, total, totalPages };
}
```

```
// Xəta:
{ result: false, data: null, message: "Xəta mesajı" }
```
```

---

## ## 13. Auth Header-ları (axios interceptor-da avtomatik)

```
```
Authorization: Bearer <access_token>
x-branch-id: <active_branch_id>
Accept-Language: az
```
```

---

## ## 14. Xidmət Qiyməti Hesablaması

```
```pseudocode
// Sadə hesablama (hazırda istifadə olunmur, gələcəkdə lazım ola bilər):
```

```
FUNCTION calcTotalAmount(price, expenses, discount):
RETURN max(0, price - expenses - discount)
```

```
FUNCTION calcExecutorAmount(price, pct):
RETURN round(price * pct / 100, 2)
```

```
FUNCTION calcReferrerAmount(price, pct):
RETURN round(price * pct / 100, 2)
```

```
// Hazırkı save axışında:
// price = service.start_price (kataloqdan)
// expenses, discount, executor_percentage, referrer_percentage → default 0
// Backend total_amount, executor_amount, referrer_amount-u özü hesablaya bilər
```
```

---

## ## 15. RBAC

```
```typescript
// Dental əməliyyatlar üçün (hazırda tətbiq edilməyib, plan):
const { hasPermission: canSave } = userRBAC([
```

```
ProfileRole.ADMIN, ProfileRole.MANAGER, ProfileRole.EMPLOYEE,
]);
```

```
const { hasPermission: canViewPrices } = userBAC([
ProfileRole.ADMIN, ProfileRole.MANAGER,
]);
```

16. Açıq Suallar (Backend ilə razılaşdırılmalı)

1. `**`business_type.slug`**` – dental üçün dəqiq slug nədir? (``dental``, ``DENTAL``, ``stomatology``?)

2. `**`createService`` – bir diş, bir xidmət, yoxsa çoxlu diş bir xidmət?
Hazırda: hər ``entries`` elementi üçün ayrıca ``createService`` çağırılır (bir diş = bir servis yazısı).

Alternativ: bütün seçilmiş dişlər bir ``tooth_numbers`` string-ə yığılıb ("26,27,18") bir servis yaradılır.

Backend hansı formatı gözləyir?

3. `**`executor_amount` / `referrer_amount`**` – Frontend hesablayıb göndərirmi, yoxsa backend ``executor_percentage``-dən özü hesablayır?

4. `**`is_completed`**` – Bu field kim tərəfindən, nə zaman ``true`` olur?

5. `**Odontogram status-ları**` – ``GET /dental/patients/:id/odontogram`` cavabı: Yalnız ``TREATED`` status var, yoxsa ``MISSING``, ``PLANNED`` da?

Frontend hazırda yalnız ``tooth_number``-i oxuyur (status-u filter etmir).

17. Implementasiya Edilib (:white_check_mark:) / Gözləyir (:hourglass_flowing_sand:)

Komponent	Status	Qeyd
Odontogram wrapper (daimi)	:white_check_mark:	<code>`teeth-{fdi}`</code> format düzəldilib
OdontogramDeciduous wrapper	:white_check_mark:	decToPerm mapping ilə
react-odontogram CSS import	:white_check_mark:	<code>`react-odontogram/style.css`</code>
DentalAPI static class	:white_check_mark:	tam CRUD
EndpointResources.dental	:white_check_mark:	patientServices əlavə edilib
useDentalSessionStore	:white_check_mark:	modal arası persistence
useOdontogramHistory + refetch	:white_check_mark:	
PatientOperationsTab	:white_check_mark:	table-first + save
Prosedur rəngləmə (teethConditions)	:white_check_mark:	PALETTE 8 rəng
Real servis kataloqu (ServicesAPI)	:white_check_mark:	getBranchServices
Business type gating	:hourglass_flowing_sand:	hələlik açıqdır
DentalReservationModal	:hourglass_flowing_sand:	gəlişmiş rezervasiya formu
Kabinet (dental room)	:hourglass_flowing_sand:	backend hazır deyil
RBAC dental üçün	:hourglass_flowing_sand:	
Dental pages / routing	:hourglass_flowing_sand:	

<!-- CABINET_PENDING

Kabinet (Dental Rooms) – backend hazır olduqdan sonra:

- EndpointResources.dental.cabinets: ``/dental/cabinets``
- EndpointResources.dental.cabinet: (id) => ``/dental/cabinets/${id}``
- GET/POST/PUT/DELETE /dental/cabinets

- IDentalAppointmentBody.cabinet_id field əlavə et
 - CabinetSelect component, cabinet.store.ts
- >